

https://youtu.be/BSVKUk58K6U?si=Q4d5tJ6JjtpJ_uL

—

create detailed notes for this youtube video. divide topics wise same as video and also include code snippets and include examples used in the video. don't forget to add important concets and points discussed in the video .

Here are detailed, **topic-wise notes** based on the video "[OOP 1 | Introduction & Concepts - Classes, Objects, Constructors, Keywords](#)" by Kunal Kushwaha, including **important concepts, explanations, code snippets, and examples** used in the video^[1].

1. Introduction to OOP in Java

- **OOP (Object-Oriented Programming)** empowers you to structure your code into reusable blueprints (classes) and create real-world-like objects.
- **Core principles of OOP:** Encapsulation, Inheritance, Polymorphism, and Abstraction.
- OOP provides improved code **organization, reusability, and maintainability**^[1].

2. Java Classes

- **Class:** A blueprint/template for creating objects. It groups **properties (fields/attributes)** and **behaviors (methods/functions)**.
- **Class Naming Convention:** Use PascalCase. Example: Student, Car.

Example: Defining a Class

```
class Student {  
    int rollNo;  
    String name;  
    float marks;  
}
```

- The above code defines a Student class with three fields: rollNo, name, marks^[1].

3. Objects

- **Object:** An instance of a class. It represents a real-world entity created using a class blueprint.
- You can create multiple objects from a single class, and each object maintains its own **state (fields/properties)**.

Creating an Object

```
Student kunal = new Student();
kunal.rollNo = 101;
kunal.name = "Kunal";
kunal.marks = 80.5f;
```

- Here, `kunal` is an object of type `Student`. The object's data is stored in the heap memory^[1].

4. Class vs Object

Class	Object
Blueprint/template	Instance (real-world entity)
No memory is allocated	Memory is allocated at runtime (heap)
One class can create many objects	Each object is unique in memory

Key Point: The class defines *structure*; object represents *data and behavior*^[1].

5. Properties of an Object

- Each object has its own **copy** of the non-static attributes.
- Modifying the properties of one object does **not affect** other objects^[1].

6. Accessing Instance Variables

- **Dot operator (.)** is used to access or modify an object's attributes.

```
System.out.println(kunal.name); // Prints: Kunal
```

- Separate memory space for each object's properties.

7. How to Create Objects

- You use the `new` keyword followed by a constructor call to generate a new object:

```
Student student1 = new Student();
Student student2 = new Student();
```

- `Student()` here calls the *constructor* of the class (default or user-defined)^[1].

8. Dynamic Memory Allocation

- All objects in Java are allocated memory in the **heap** area at runtime when you use the `new` keyword.
- The variable (like `kunal`) holds a **reference/address** to the actual object in memory^[1].

9. Manipulating Objects

- You can change fields by assigning new values using dot notation:

```
student1.name = "Abhishek";  
student1.marks = 93.0f;
```

- Each object's data is independent.

10. Java Constructors

- **Constructor:** Special method that gets called when an object is created. Used to initialize the object's state.
- **Default Constructor:** If you don't define any constructor, Java provides a no-argument constructor by default^[1].

Example of User-Defined Constructor

```
class Student {  
    int rollNo;  
    String name;  
    float marks;  
  
    // Constructor  
    Student(int roll, String nm, float mks) {  
        rollNo = roll;  
        name = nm;  
        marks = mks;  
    }  
}  
  
Student student3 = new Student(104, "Rahul", 71.8f);
```

11. 'this' Keyword

- Used within a class to refer to the current object instance.
- Resolves ambiguity when parameter names are the same as field names.

Example

```
class Student {  
    int rollNo;  
    String name;  
  
    Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
}
```

- `this.rollNo` refers to the field, while `rollNo` is the constructor parameter^[1].

12. Constructor Overloading

- You can have **multiple constructors** in a class, each with different parameters (overloading).
- Allows flexible object creation.

Example

```
class Student {
    int rollNo;
    String name;

    // No-argument constructor
    Student() {
        this.rollNo = 0;
        this.name = "Unknown";
    }

    // Parameterized constructor
    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
}
```

13. Calling One Constructor from Another ('this()')

- Use the `this()` syntax to call another constructor in the same class to avoid duplication.

Example

```
class Student {
    int rollNo;
    String name;

    Student() {
        this(999, "Default");
    }

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
}
```

14. Why 'new' Keyword is Not Used for Primitives

- **Primitive types** (`int`, `float`, etc.) are *not objects*; they are simple values directly stored on the stack.
- Using `new` is only for *reference types* (objects) ^[1].

15. Memory Allocation of 'new' Keyword

- The `new` keyword **allocates memory for the object** on the heap and returns the reference address.
- Multiple variables can refer to the same object reference in memory.

16. Wrapper Classes

- Java provides *wrapper classes* for all primitive types, enabling them to be treated as objects (for collections, etc.)
- Examples:
 - `int` → `Integer`
 - `float` → `Float`

Example

```
int num = 10;  
Integer numObj = Integer.valueOf(num);
```

17. 'final' Keyword

- Used to make variables unchangeable (constants), methods non-overridable, and classes non-inheritable.

Usage

```
final int MAX_STUDENTS = 100;
```

- Value can't be changed once assigned.

18. Garbage Collection

- Java automatically deletes objects that no longer have any references, freeing up heap memory.
- Developers don't need to delete objects manually.

19. Summary of Additional Points

- OOP in Java provides a framework for building complex applications with greater code reuse and clarity.
- Real-world modeling: A class captures generic features; objects capture actual data.
- Clarity between **reference types** (objects) and **primitives** (simple values).
- **Access modifiers, encapsulation, inheritance, and polymorphism** are covered in further videos.

All main code examples, explanations, and OOP basics presented here follow the flow and content as outlined by Kunal Kushwaha in his video^[1].

*
**

1. <https://www.youtube.com/watch?v=BSVKUk58K6U>